Winter 2026

Final Project Report

Rube Goldberg Machine: The Electronic Chain Reaction

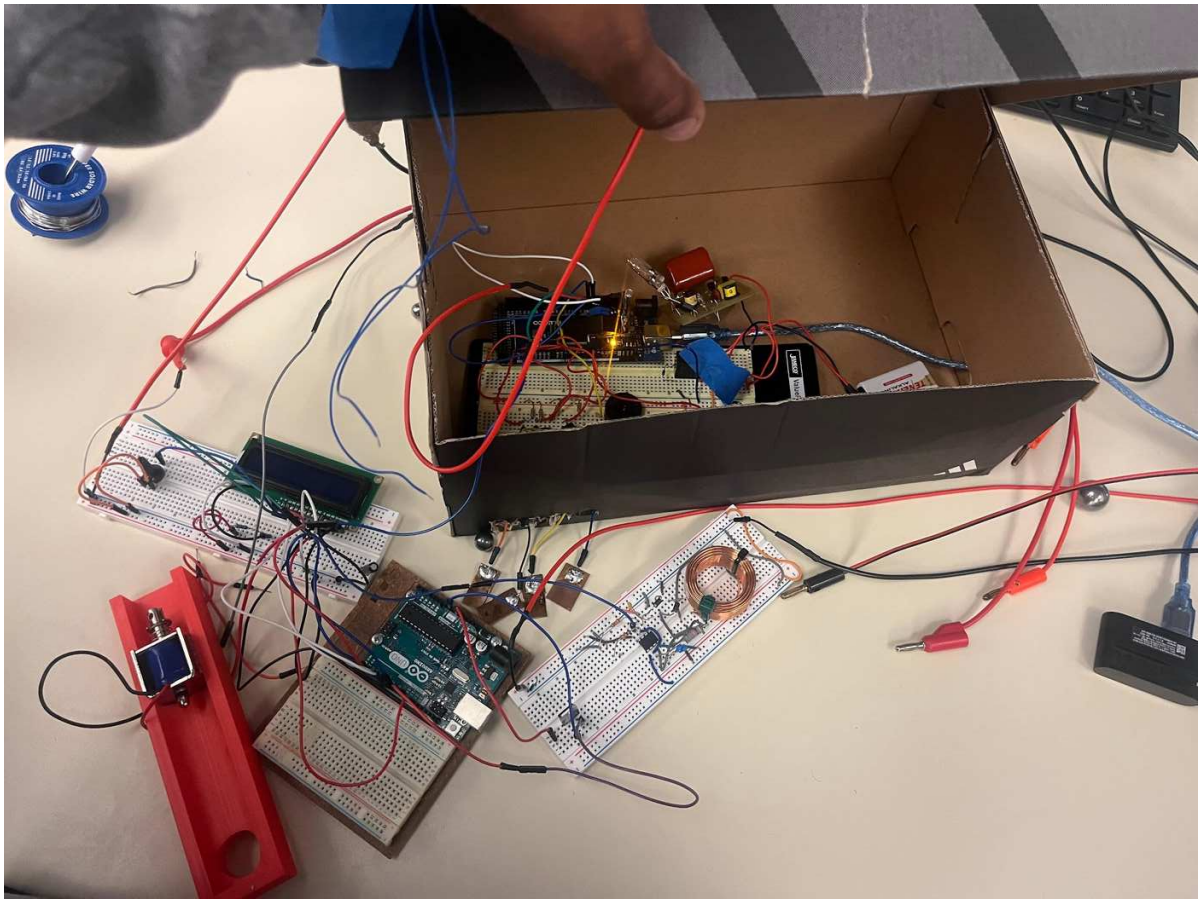


Figure 1: Complete RGM Setup

Arjanveer Samra

March 10, 2026

1. Explanation of RGM Steps

This section describes how each circuit stage of the RGM works and how they connect to accomplish the final task. The RGM consists of nine sequential stages, beginning with a capacitive piano key sequence and ending with an LCD displaying 2 rows of 16 white blocks each.

Block Diagram

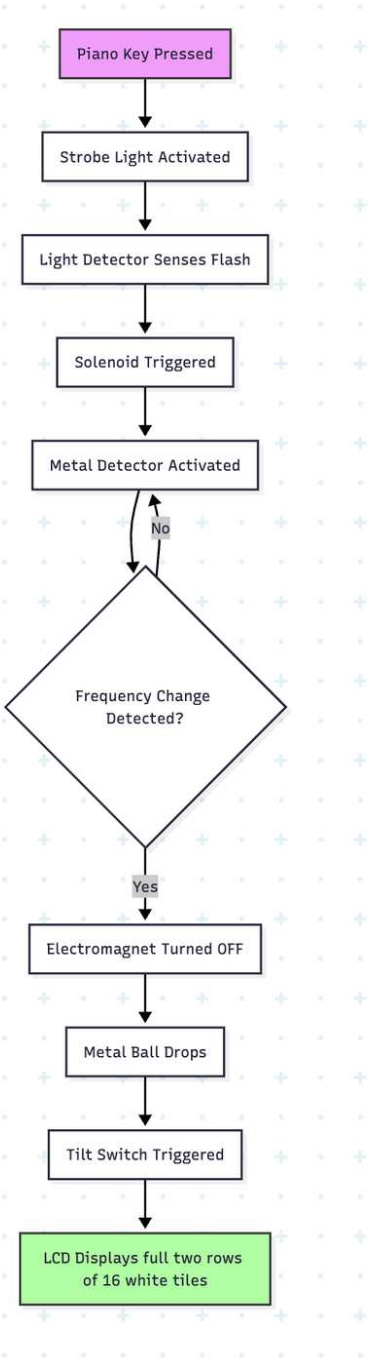


Figure 1. RGM Step-by-Step Block Diagram

Step 1: Capacitive Touch Arduino Piano (Lab 4)

What it does:

The capacitive piano is the input trigger for the entire RGM. The user must input the correct 5-note sequence (keys 1, 3, 2, 4, 1) using four touch pads made of copper foil connected to an Arduino Mega.

How it works:

Each touch pad consists of an RC circuit with a 2.2 MΩ resistor. When the finger touches the touch pad, the capacitance increases, thus increasing the time taken to accumulate the required charge according to the CapacitiveSensor library of Arduino. When the capacitance exceeds 1000 counts, the key is pressed. A unique tone, such as C4, D4, E4, or F4, is played for each key using a buzzer connected to the Arduino. After the correct sequence of 5 notes, the relay connected to pin 8 of the Arduino is activated.

Step 2: Relay — Connects Piano to Strobe Light (Labs 2 & 4)

What it does:

The relay provides electrical isolation between the low-voltage Arduino circuit (5V) and the higher-voltage strobe light circuit (9V).

How it works:

The Arduino switches the relay using a BC548 NPN transistor switch to connect to a magnetic field inside the relay, which in turn closes the normally open contacts to connect the 9V supply to the strobe light circuit. A flyback diode is used to protect the transistor from voltage spikes from the inductive nature of the relay coil.

Step 3: Strobe Light (Lab 5)

What it does:

The strobe light produces a bright flash of light when powered by the relay, which is then used to trigger the light detector circuit.

How it works:

The strobe circuit consists of three parts: (1) An RC oscillator with transistor switches Q1 and Q2 that drives a step-up transformer to generate high voltage (~580V), (2) a rectifier/capacitor section that stores the high voltage energy, and (3) a trigger circuit with an SCR and a trigger transformer to quickly discharge the stored energy into a xenon flash tube. The flash rate is determined by R3 and C3 (τ = R3 x C3).

Step 4: Light Detector with Schmitt Trigger (Lab 2)

What it does:

The light detector senses the strobe flash and activates the solenoid.

How it works:

A photoresistor and a potentiometer VR1 form a voltage divider circuit known as V2. Another fixed voltage divider circuit is used to generate a reference voltage known as V1. Finally, an LM324 op-amp is used to compare V2 and V1. If V2 is greater than V1, it means there is light detection. The circuit includes a potentiometer known as VR2 to provide hysteresis to the circuit to prevent oscillations. Finally, a BC548 transistor is used to switch on a solenoid relay.

Step 5: Solenoid — Launches Steel Marble (Lab 7)

What it does:

The solenoid acts as a mechanical actuator that physically launches the steel marble into the metal detector coil when triggered by the light detector.

How it works:

When the current passes through the solenoid coil, the magnetic field will attract the inner plunger, which will then push the steel ball down the track and into the inductor coil of the metal detector. The solenoid will be secured in the track piece, which has dimensions of 6 in x 2 in x 1 in with a cylindrical cutout of 0.75 in.

Step 6: 555 Timer Metal Detector (Lab 6)

What it does:

The metal detector senses the presence of the steel marble inside the inductor coil by detecting a drop in oscillation frequency, then signals the Arduino to turn off the electromagnet.

How it works:

A NE555 timer is set to astable mode with an LC oscillator circuit. Without any metal, the circuit runs at 8,760 Hz. Once the steel marble is introduced to the inductor circuit, it increases the inductance of the circuit. This reduces the frequency to 8,310 Hz, or a 5.1% change above the required 4% minimum. The Arduino on pin 12 reads the frequency with pulseIn() and makes the electromagnet pin LOW when it drops by 4% or more.

Step 7: Electromagnet Turns OFF — Ball Released (Lab 7)

What it does:

The electromagnet holds the steel ball in place above the tilt switch. When the metal detector detects the marble in the coil, it cuts power to the electromagnet, releasing the ball.

How it works:

A MOSFET, connected to Arduino pin 11, is used to control the current passing through the electromagnet coil. The electromagnet is initially energized (HIGH), holding the ball. The Arduino detects the ≥4% frequency drop and sets pin 11 LOW, switching off the electromagnet. The MOSFET needs to be connected to a shared ground between the Arduino and the 9V supply. See Section 2 for more details and the schematic diagram.

Step 8: Metal Ball Drops onto Tilt Switch

What it does:

Once released by the electromagnet, the steel ball falls under gravity onto the tilt switch, physically actuating it.

How it works:

The tilt switch is normally open and closes when it is tilted beyond a certain point. The steel ball falls on the tilt switch and actuates it, thereby closing the switch and sending a signal to the Arduino board to update the LCD display.

Step 9: LCD Displays 2 rows of 16 white blocks each — Last Step

What it does:

The LCD display outputs a confirmation message when the tilt switch is triggered, completing the RGM sequence.

How it works — see Section 2 for full detail.

2. Last Step — Tilt Switch & LCD Display

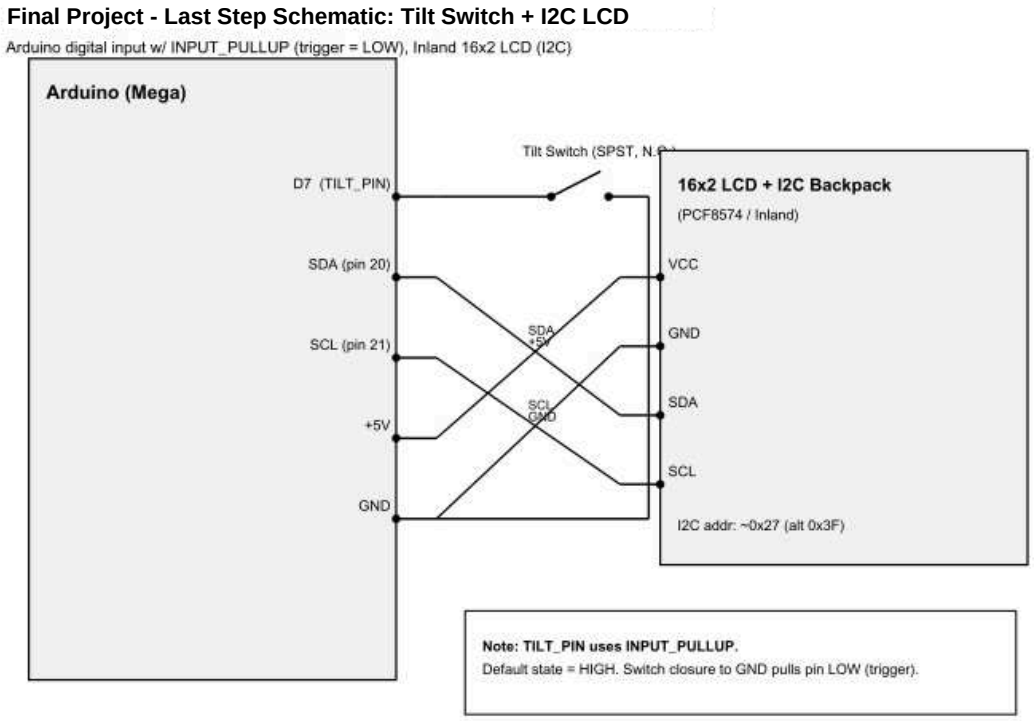
The final step of the RGM is the tilt switch and LCD display circuit. This is the chosen step by the team as it is part of the RGM and is the last step of the student-designed RGM. Once the steel ball is released from the electromagnet and lands on the tilt switch, 2 rows of 16 white blocks each are displayed on an LCD to indicate that the RGM has been successfully completed.

Description

The tilt switch is connected to the Arduino board as a digital input. Once the ball falls and tilts the switch, the digital pin is normally HIGH and goes LOW when the tilt switch closes to ground.. The Arduino board then sends the signal to display 2 rows of 16 white blocks each to the I2C or parallel LCD display. This is a clear and unambiguous visual indication that all steps of the RGM have been fired correctly.

How It Works

The tilt switch is connected between the Arduino pin and the GND pin, and the internal pull-up resistor on the pin is enabled (INPUT_PULLUP). The state of the pin will be high in the default state, and when the ball falls and tilts the switch, the state will be low, triggering the LCD output. The Arduino code will poll the state of the pin and write to the LCD when the state changes from high to low.



Arduino Code — Last Step

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Pins
#define FREQ_PIN 8
#define MAG_PIN 9
#define TILT_PIN 7

// Inland I2C LCD (most common address is 0x27, sometimes 0x3F)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Metal detector settings
const float BASE_FREQ = 8600.0;
const float TRIP_DELTA = 150.0;
const int STREAK_N = 5;

const unsigned long RELEASE_DELAY = 600;
const unsigned long MAG_OFF_TIME = 1500;

const unsigned long DEBOUNCE = 50;
const unsigned long TILT_WINDOW = 5000;

unsigned long onT, offT;

int streak = 0;
bool pending = false;
unsigned long pendingStart = 0;

bool magOff = false;
unsigned long magStart = 0;

bool lastTilt = HIGH;
bool stableTilt = HIGH;
unsigned long tiltTimer = 0;

bool tiltArmed = false;
bool tiltDone = false;
unsigned long tiltStart = 0;

bool lcdFilled = false;


// read frequency from 555 timer
float readFreq() {
  onT = pulseIn(FREQ_PIN, HIGH, 30000UL);
  offT = pulseIn(FREQ_PIN, LOW, 30000UL);

  if (!onT || !offT) return -1;
```

```
  unsigned long period = onT + offT;
  if (period < 20 || period > 5000) return -1;

  return 1000000.0 / period;
}


// fill LCD with 2 rows of 16 blocks
void fillLCD() {
  if (lcdFilled) return;

  byte block[8] = {0x1F,0x1F,0x1F,0x1F,0x1F,0x1F,0x1F,0x1F};
  lcd.createChar(0, block);

  lcd.clear();
  lcd.setCursor(0,0);
  for (int i=0; i<16; i++) lcd.write((byte)0);

  lcd.setCursor(0,1);
  for (int i=0; i<16; i++) lcd.write((byte)0);

  lcdFilled = true;
}

void clearLCD() {
  if (!lcdFilled) return;
  lcd.clear();
  lcdFilled = false;
}


// tilt switch debounce (INPUT_PULLUP so LOW = triggered)
void updateTilt() {
  bool r = digitalRead(TILT_PIN);

  if (r != lastTilt) {
    tiltTimer = millis();
    lastTilt = r;
  }

  if (millis() - tiltTimer > DEBOUNCE) {
    if (stableTilt != r) {
      stableTilt = r;

      if (stableTilt == LOW && tiltArmed &&
          millis() - tiltStart <= TILT_WINDOW) {
        tiltDone = true;
      }
    }
  }
```

```
  if (tiltArmed && millis() - tiltStart > TILT_WINDOW) {
    tiltArmed = false;
  }
}


void setup() {
  pinMode(FREQ_PIN, INPUT);
  pinMode(MAG_PIN, OUTPUT);
  pinMode(TILT_PIN, INPUT_PULLUP);

  digitalWrite(MAG_PIN, HIGH);   // hold ball at start

  Serial.begin(9600);

  lcd.init();
  lcd.backlight();
  lcd.clear();
}


void loop() {

  updateTilt();

  // final step
  if (tiltDone) {
    fillLCD();
    digitalWrite(MAG_PIN, LOW);
    delay(50);
    return;
  } else {
    clearLCD();
  }

  // magnet off timing
  if (magOff) {
    digitalWrite(MAG_PIN, LOW);

    if (millis() - magStart >= MAG_OFF_TIME) {
      magOff = false;
      streak = 0;
      pending = false;

      digitalWrite(MAG_PIN, HIGH);

      tiltArmed = true;
      tiltStart = millis();
    }

    delay(20);
    return;
```

```
  }

  digitalWrite(MAG_PIN, HIGH);

  float f = readFreq();

  if (f > 0) {
    Serial.println(f);

    if (f < BASE_FREQ - TRIP_DELTA) {
      streak++;
      if (streak >= STREAK_N && !pending) {
        pending = true;
        pendingStart = millis();
      }
    } else {
      streak = 0;
      pending = false;
    }
  } else {
    streak = 0;
    pending = false;
  }

  // release ball after delay
  if (pending && millis() - pendingStart >= RELEASE_DELAY) {
    magOff = true;
    magStart = millis();
    digitalWrite(MAG_PIN, LOW);
    pending = false;
  }

  delay(20);
}
```

3. Discussion

This section addresses any RGM steps that did not work as expected, the potential causes, and the fixes applied during development and integration.

Issue 1: Strobe Light Not Flashing (Lab 5 / Lab 7 Integration)

During the system integration, the strobe light did not flash, despite the proper operation of the relay. Debugging the problem, we realized that one of the transistors in the oscillator stage of the

strobe PCB had a broken lead. Since the transistor drives the step-up transformer, the high voltage charging stage did not operate.

Replacing the transistor and making sure it was correctly oriented, the oscillator began to operate, and the strobe light began to flash.

Issue 2: Light Detector Triggered by Ambient Light (Lab 7)

The first issue to deal with was that the light detector circuit was actually being activated by the lighting in the room, rather than the flash from the strobe. This is because the photoresistor and VR1 form a voltage divider, and so the reference voltage was too low.

The value of VR1 was adjusted to increase the threshold of the light detection circuit so that only the high-intensity flash from the strobe would cause V2 to be greater than V1. Once this had been done, the Schmitt trigger circuit would prevent any oscillation and would not trigger from the ambient light.

Issue 3: Electromagnet Releasing Immediately After Piano Key Press

In the early stages of integration, the electromagnet would release the steel ball immediately after pressing a piano key, rather than after metal detection.

The problem was due to the detection of small fluctuations in the frequency of the 555 oscillator by the Arduino before the steel ball entered the inductor coil. The release of the electromagnet was based on a threshold value, and small noises were detected by the Arduino.

To achieve this:

A streak counter (STREAK_N) was implemented to ensure multiple consecutive low-frequency readings were taken before release.

A delay (RELEASE_DELAY) was introduced before turning off the magnet.

The threshold was adjusted to ensure that there was a consistent change with respect to the baseline frequency.

This ensured that the ball did not fall prematurely and only fell when actual metal detection was made.

Issue 4: MOSFET Not Switching Reliably (Lab 7)

The electromagnet MOSFET initially failed to switch correctly under the control of the Arduino. This problem, however, was caused by the lack of common ground between the Arduino, which had its 5V supply from the USB, and the external 9V supply for the electromagnet.

The lack of common ground essentially meant there was no definite gate-source voltage, hence the inability of the MOSFET to function correctly. Once both grounds were connected, the MOSFET functioned correctly, and the electromagnet released the ball correctly.

Issue 5: Metal Detector Threshold Tuning (Lab 6 / Lab 7)

The frequency of the 555 timer LC oscillator was approximately 8.7 kHz without the presence of metal. Once the steel marble was introduced into the coil, the frequency decreased to approximately 8.3 kHz, which is about a 5% reduction.

Instead of using the percent change calculation, a set threshold was used that represented approximately a 4% reduction from the baseline.

This was accomplished by adjusting the TRIP_DELTA value until false triggers were eliminated and the steel ball was still detected.

Issue 6: RC Time Constant Discrepancy (Lab 4)

In Lab 4, the percent difference between the measured charging time constant and the theory value was 38.5%, and the percent difference between the measured discharging time constant and the theory value was 12.8%. These differences are due to the following factors:

- Oscilloscope Cursor Positioning Error
- Tolerance of the 39 kΩ resistor and the 0.02 μF capacitor
- Breadboard Contact Resistances

When the resistor value was changed to 20 kΩ, the percent error decreased, which indicates that the percent error increases with the resistor value.

Steps Working as Expected During Demonstration

- Once debugging and calibration were complete, all nine RGM stages were working properly for in-class demonstration:

- Capacitive piano successfully recognized the proper sequence of 5 notes
- Relay successfully switched 9V power supply
- Strobe successfully flashed in sequence
- Light detector only flashed in response to strobe light
- Solenoid successfully launched steel ball
- 555 metal detector successfully changed frequency by 5%
- Electromagnet only released after successful detection
- Tilt switch successfully switched (LOW trigger with INPUT_PULLUP)
- LCD displayed 2 rows of 16 white blocks after successful completion

4. End-of-Life Lab Kit

This section describes our plan to responsibly reuse or recycle the components of the RGM system after the course concludes.

Component Reuse Plan

- Arduino Mega: This will be retained and used for future personal electronics, home automation, or additional EE courses.
- Breadboards and Jumper Wires: These will be stored and used for future prototyping in additional labs and personal projects.
- Capacitive touch pads (copper foil): These will be retained and used as touch sensors for additional Arduino projects or as general-purpose conductive pads.
- Relay Module: This will be retained and used for future projects requiring control of low voltage, high voltage devices (e.g., home automation switches).
- LM324 Op-Amp IC's: These will be retained and used for additional analog circuit projects requiring comparators or signal conditioning circuits.
- NE555 Timer IC: This will be stored and used for future timing circuit or oscillator projects.
- LCD Display Module: This will be retained and used for additional Arduino display projects.
- Tilt Switch: This will be stored and used for additional motion or orientation detection circuits.
- Passive components (Resistors, Capacitors, Inductors): These will be stored and organized in a component organizer for future use.
- BC548 Transistors and 1N4148 Diodes: These will be retained and used as switches and protection diodes for future circuits.
- PCB from Lab 3 (OSH Park): This will be retained and used as a PCB reference and potentially reused if the LM324 light detector circuit is needed again.

Recycling Plan

- Strobe light PCB: This will be recycled at an e-waste recycling center because of the hazardous materials used.
- 9V Batteries: These will be recycled at a drop-off location, which can be found at most hardware stores and Best Buy.
- Broken or Non-Functional Parts: These will be recycled at an e-waste collection event, instead of being discarded in the regular trash.
- Copper wire used in the inductor coil: This can be recycled or repurposed.

Environmental Commitment

We are committed to not disposing of any of these electronic components in general household waste. All ICs, PCBs, and batteries will be sent to appropriate e-waste disposal facilities. The

reusable components will be retained for future use in educational projects among all three team members, eliminating the need to buy additional components for future projects.

5. Video Demonstration

VIDEO DEMONSTRATION LINKED HERE

This video shows all nine steps of the RGM in operation as they are demonstrated in class on March 10, 2026. All members of the group are shown on camera and give their names. All steps are shown in working order as described in Step 1. The final step shows 2 rows of 16 white blocks each on the LCD display.